Pós Graduação
Data Science e Big Data na Indústria

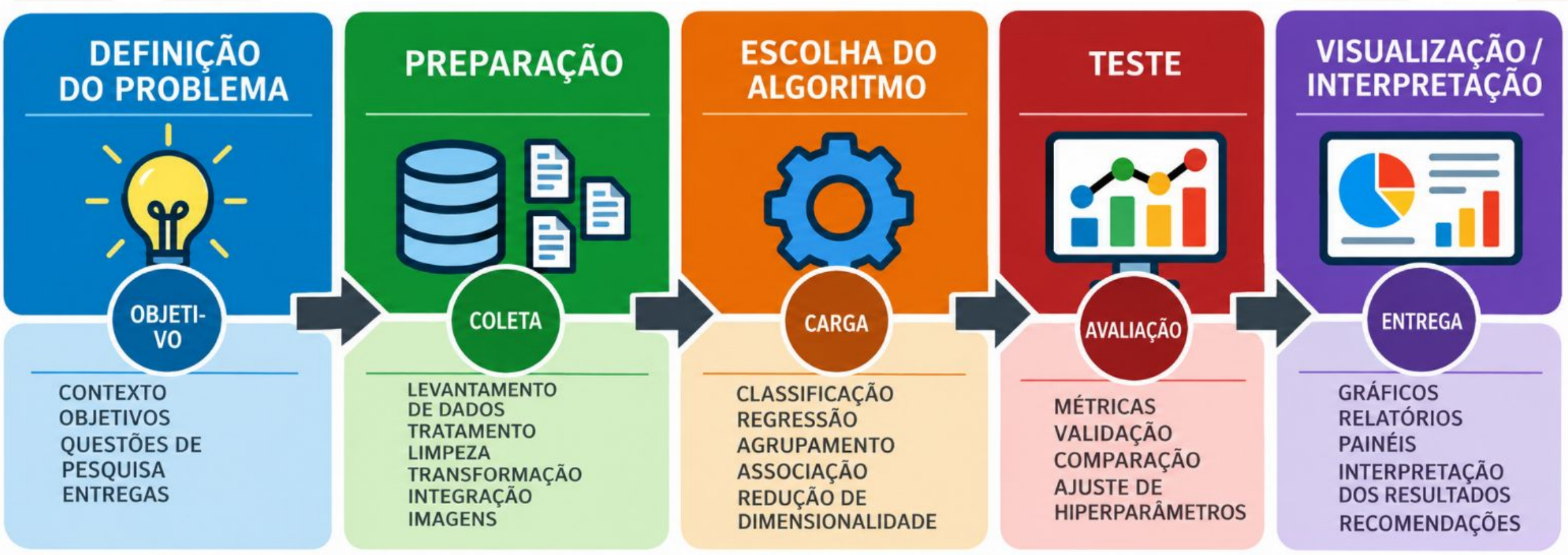
Disciplina: Data Science e Big Data

Aula 04

Prof. Me Daniel Vieira

UNISENAI
Inovação no Ensino Superior para a Indústria

Etapas de um projeto de Data Science



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Base de dados utilizada

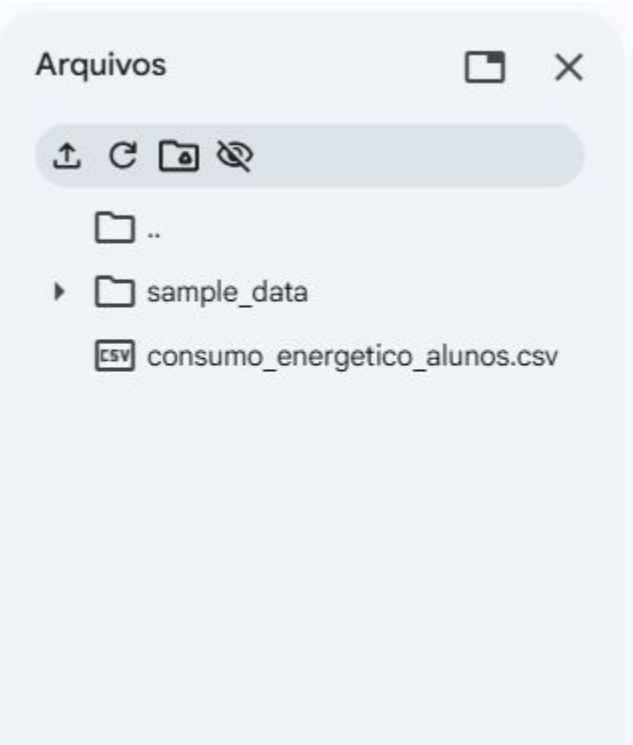
Aula 03 - Pós Data Science e Big Data - Exploração dos dados Objetivos: Detectar problemas de qualidade dos dados, preparar os dados para modelos de Machine Learning, treinar uma Árvore de Decisão, intepretar métricas e aplicar um segundo classificador ao conceito de eficiência energética

```
# Importando bibliotecas
import numpy as np # permite trabalhar com array
import pandas as pd # permite importar a base de dados
import matplotlib.pyplot as plt # Permite plotar gráficos
from sklearn.model_selection import train_test_split # Permite dividir a base de dados em treino e teste
from sklearn.impute import SimpleImputer # Permite tratar dados ausentes na base de dados
from sklearn.pipeline import Pipeline # Permite criar um fluxo nas etapa de analise de dados
from sklearn.compose import ColumnTransformer # Permite aplicar transformações de dados diferentes para colunas diferentes
from sklearn.preprocessing import OneHotEncoder # Permite converter variáveis categóricas em um forma numérico para que os modelos de Machine Learning entendam
from sklearn.tree import DecisionTreeClassifier, plot_tree #Importa o modelo de árvore de decisao
from sklearn.metrics import(accuracy_score, precision_score, recall_score, f1_score,classification_report, ConfusionMatrixDisplay)
import seaborn as sns
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Carregar a base de dados no Google Colab




```
1 # Importando a base de dados
2 df = pd.read_csv('consumo_energetico_alunos.csv')
```

```
1 df.head()
```

	timestamp	turno	producao_unidades	temperatura_ambiente_c	maquinas_ativas	horas_operacao_turno	consumo_kwh
0	2026-10-12 22:00:00	noite	1102.0	21.80	7.0	4.86	283.36
1	2027-11-15 14:00:00	tarde	1356.0	21.31	6.0	6.90	335.73
2	2028-12-27 06:00:00	manha	796.0	28.89	3.0	6.74	197.15
3	2027-04-25 14:00:00	tarde	1406.0	22.52	7.0	6.05	291.80
4	2028-03-27 06:00:00	manha	1431.0	23.98	7.0	5.44	269.41

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
1 print(f"Quantidade de linhas {len(df)} | Quantidade de colunas {len(df.columns)}")
2
```

```
Quantidade de linhas 5050 | Quantidade de colunas 8
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```python
# Exploração dos dados
'''
Quantas linhas e colunas ? Quais variáveis são numéricas ? Quais são as classes de turno ? Há dados ausentes e repetidos ?
'''
print("Tipos de dados")
print(df.dtypes) # Exibe os tipos de dados das colunas
print('\n Ausentes')
print(df.isna().sum()) # Verificando dados ausentes
print('\n Duplicados: ',df.duplicated().sum()) # Verificando dados duplicados
print("\n Distribuição de turnos")
print(df['turno'].value_counts())
display(df.select_dtypes(include="number").describe().round(2))
```

```
Tipos de dados
timestamp                 object
turno                     object
producao_unidades        float64
temperatura_ambiente_c   float64
maquinas_ativas          float64
horas_operacao_turno     float64
consumo_kwh              float64
dtype: object

 Ausentes
timestamp                 0
turno                     0
producao_unidades        51
temperatura_ambiente_c   50
maquinas_ativas          50
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5050 entries, 0 to 5049
Data columns (total 7 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   timestamp              5050 non-null   object 
 1   turno                  5050 non-null   object 
 2   producao_unidades      4999 non-null   float64
 3   temperatura_ambiente_c 5000 non-null   float64
 4   maquinas_ativas        5000 non-null   float64
 5   horas_operacao_turno   4999 non-null   float64
 6   consumo_kwh            5050 non-null   float64
dtypes: float64(5), object(2)
memory usage: 276.3+ KB
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Limpeza e análise de outliers

Remova duplicatas exatas; mantenha um registro de quantas foram removidas. Converta datas sem usá-las diretamente como preditores nesta introdução. Investigue valores improváveis antes de removê-los. O limite de 50 °C para temperatura ambiente é apenas uma regra didática de sinalização, não uma especificação do processo. Importante: valores ausentes dos preditores serão preenchidos com medianas aprendidas somente no treinamento, via Pipeline. Não calcular fillna(median()) no dataset completo antes do split.

```
# Limpeza e análise de outliers
df1 =df.drop_duplicates().copy() # varre o dataframe em busca de valores duplicados
df['timestamp'] = pd.to_datetime(df['timestamp'],errors='coerce')
print('Removidas:', len(df)-len(df1),"duplicadas")
print('Datas inválidas:', df['timestamp'].isna().sum())
print('Temperaturas acima de 50 ºC para investigar:',(df['temperatura_ambiente_c']>50).sum())
print('Consumo<=0',(df["consumo_kwh"]<=0).sum())

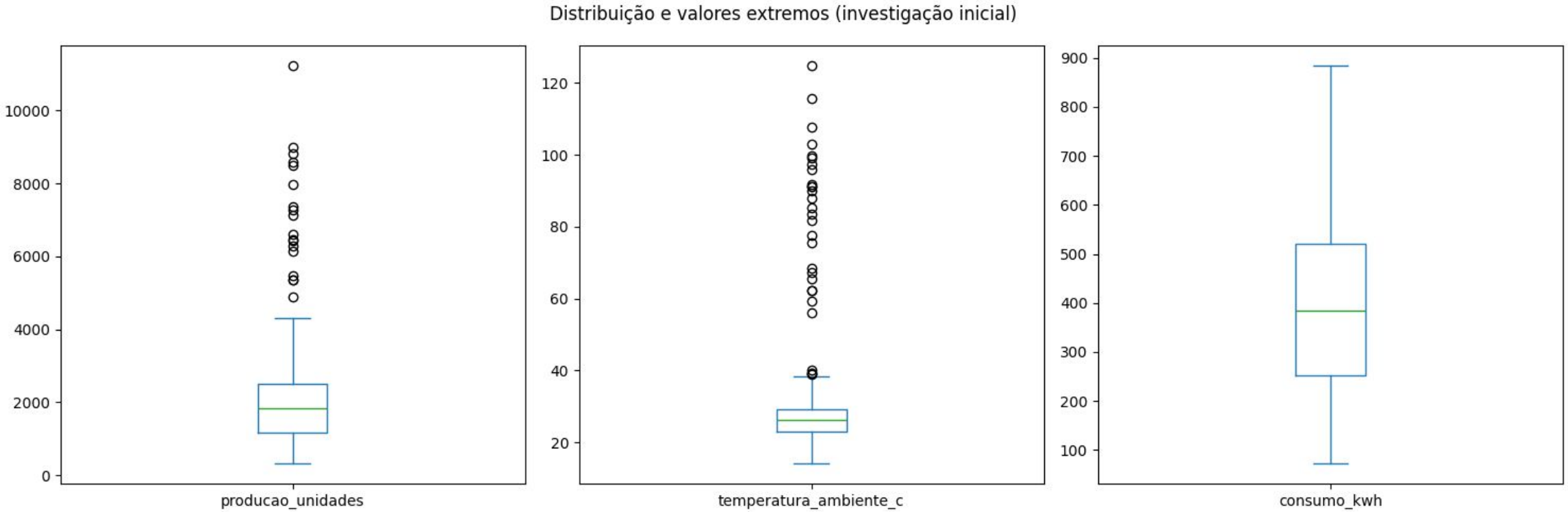
```

```
Removidas: 50 duplicadas
Datas inválidas: 0
Temperaturas acima de 50 ºC para investigar: 25
Consumo<=0 0
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

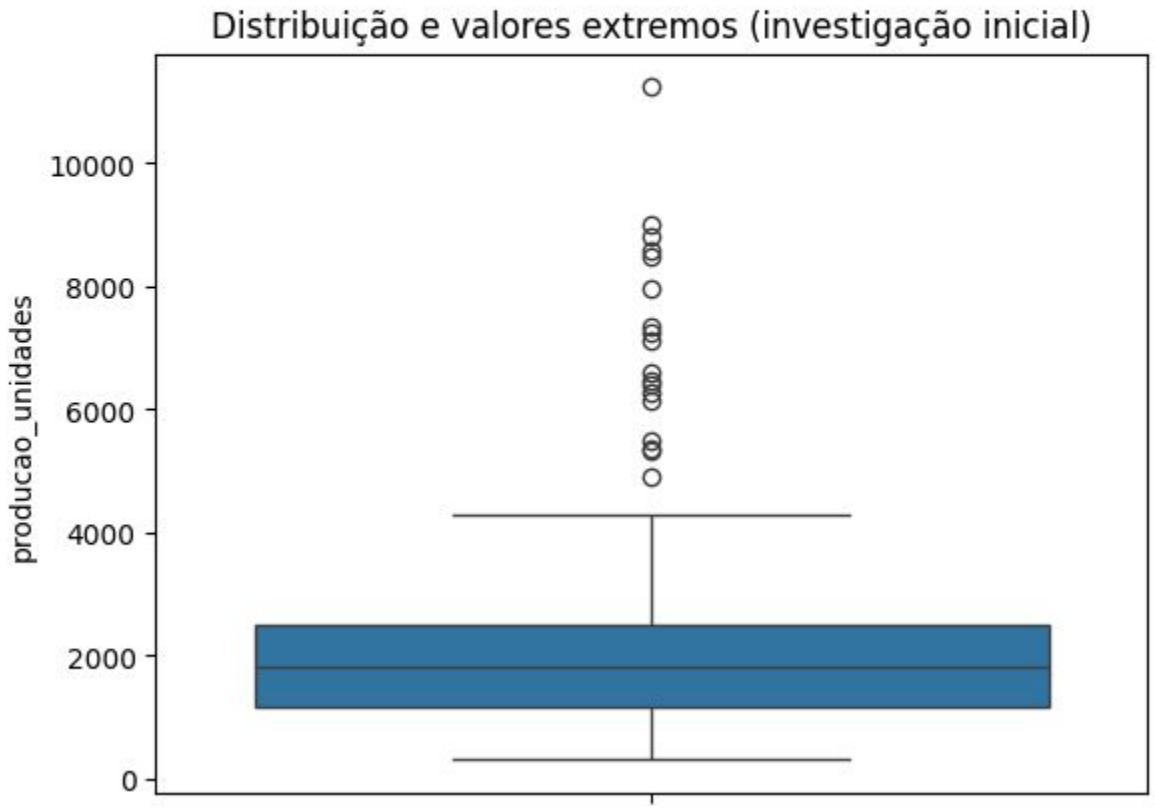
```
#Visualização
df[['producao_unidades','temperatura_ambiente_c','consumo_kwh']].plot(kind='box',subplots=True,layout=(1,3),figsize=(15,5),sharey=False)
plt.suptitle('Distribuição e valores extremos (investigação inicial)')
plt.tight_layout()
plt.show()
```



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
1 sns.boxplot(data=df['producao_unidades'])
2 plt.title('Distribuição e valores extremos (investigação inicial)')
3 plt.show()
```



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Introdução ao aprendizado supervisionado - Prever uma classe consumo alto ou baixo - Classificação: prever uma classe (consumo alto ou baixo).

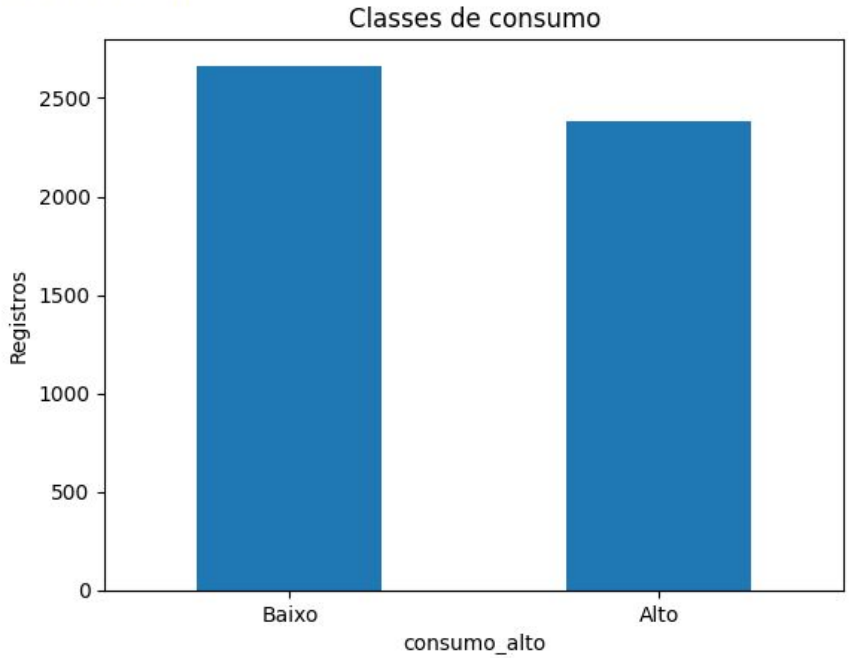
- Regressão: prever um número (ex.: kWh).
- Supervisionado: o rótulo correto está disponível durante o treino.
- Não supervisionado: buscamos padrões sem rótulos previamente definidos. Nesta atividade, alto consumo significa consumo_kwh > 400, e baixo significa <= 400. 400 kWh é um limite DIDÁTICO, não um critério de desempenho industrial. Na entrada do modelo, nunca incluir consumo_kwh, porque ele é a própria origem do rótulo.

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```python
# Consumo é medido e serve apenas para criar o rótulo histórico
assert df["consumo_kwh"].notna().all()
df["consumo_alto"] = (df["consumo_kwh"] > 400).astype(int)
print(df["consumo_alto"].value_counts().rename(index={0:"Baixo",1:"Alto"}))
df["consumo_alto"].map({0:"Baixo",1:"Alto"}).value_counts().plot(kind="bar", title="Classes de consumo")
plt.xticks(rotation=0); plt.ylabel("Registros"); plt.show()
```

```
consumo_alto
Baixo    2666
Alto     2384
Name: count, dtype: int64
```



Exemplo 2 de Machine Learning e tratamento dos dados

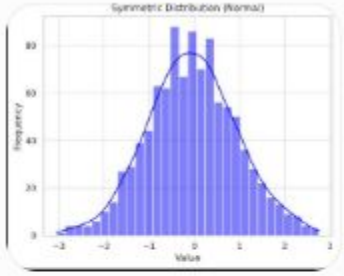
Tratamento dos dados ausentes

1. Quando utilizar média ou mediana?

Critério	Média (`mean`)	Mediana (`median`)
Distribuição aproximadamente simétrica	Recomendada	Também pode ser usada
Presença de outliers	Sensível	Mais robusta
Distribuição assimétrica	Pode ser distorcida	Geralmente preferível
Exemplo industrial	Temperatura estável	Consumo com picos extremos

Exemplo 2 de Machine Learning e tratamento dos dados

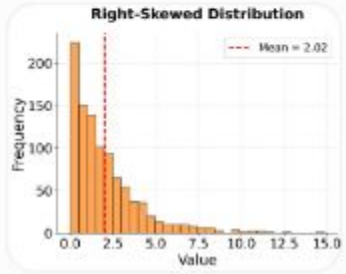
Tratamento dos dados ausentes



Distribuição aproximadamente simétrica

Os valores se concentram no centro. Média e mediana costumam ser próximas.

A média pode ser uma boa escolha para imputação.



Distribuição assimétrica

Os dados apresentam uma cauda mais longa para um dos lados.

A mediana geralmente é preferível, pois sofre menos influência de valores extremos.

Exemplo 2 de Machine Learning e tratamento dos dados

Verificando a distribuição dos dados

```python
import matplotlib.pyplot as plt

# Seleciona a variável
dados = df["producao_unidades"].dropna()

# Calcula média e mediana
media = dados.mean()
mediana = dados.median()

# Plota o histograma
plt.figure(figsize=(9, 5))

plt.hist(
    dados,
    bins=20,
    color="skyblue",
    edgecolor="black"
)

# Linha indicando a média
plt.axvline(
    media,
    color="red",
    linestyle="--",
    label=f"Média: {media:.2f}"
)

```

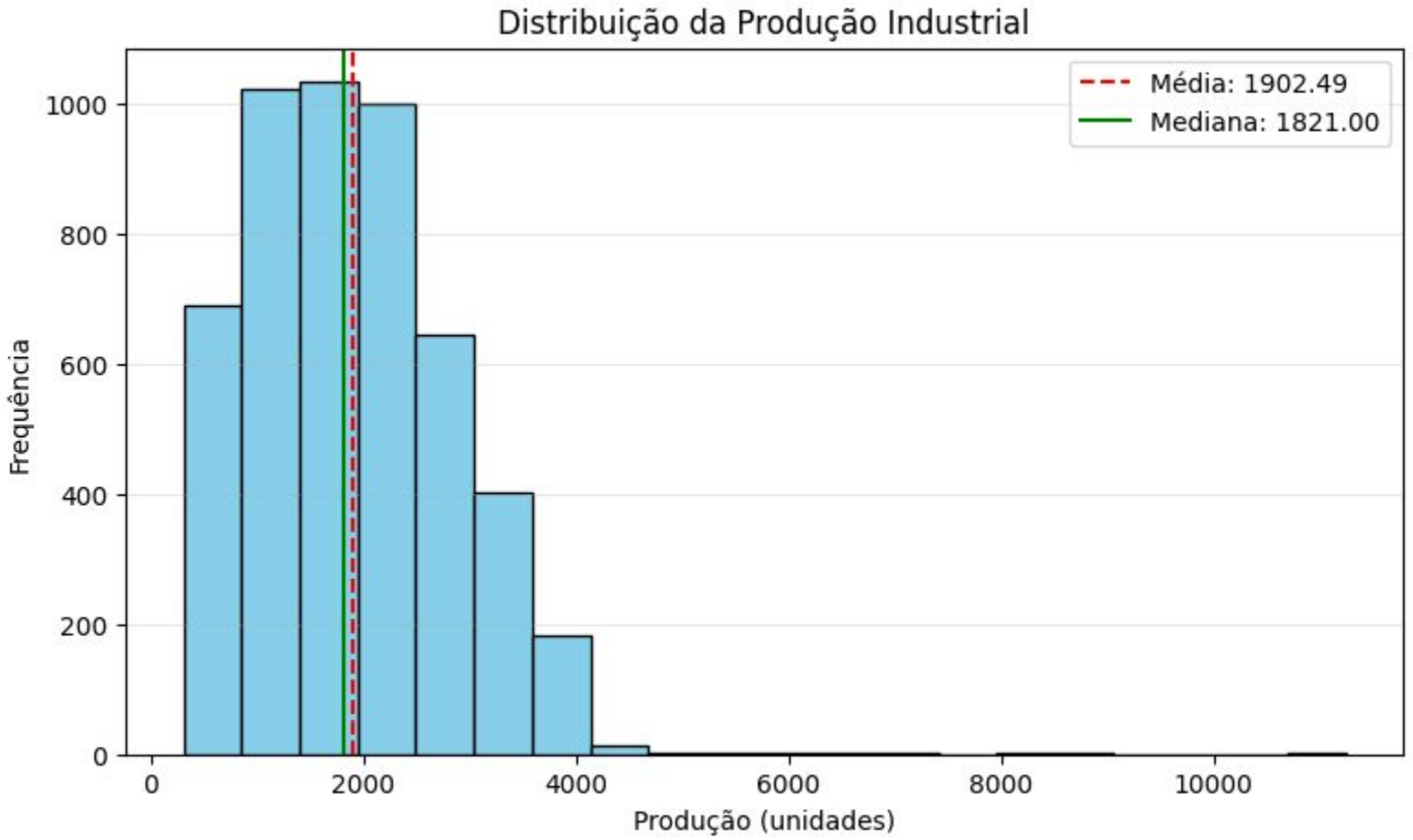
Exemplo 2 de Machine Learning e tratamento dos dados

Verificando a distribuição dos dados

```
27
28 # Linha indicando a mediana
29 plt.axvline(
30     mediana,
31     color="green",
32     linestyle="-",
33     label=f"Mediana: {mediana:.2f}"
34 )
35
36 plt.title("Distribuição da Produção Industrial")
37 plt.xlabel("Produção (unidades)")
38 plt.ylabel("Frequência")
39 plt.legend()
40 plt.grid(axis="y", alpha=0.3)
41
42 plt.show()
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Separar os atributos, treinar e prever Os preditores numéricos são produção, temperatura, máquinas ativas e horas de operação; turno é categórico. O ColumnTransformer aplica medianas aos dados numéricos e codifica o turno; o Pipeline aprende tudo sem acessar os valores do conjunto de teste.

[53] ✓ 0s

```
numericas = ["producao_unidades", "temperatura_ambiente_c", "maquinas_ativas", "horas_operacao_turno"]
categoricas = ["turno"]
atributos = numericas + categoricas
X = df[atributos]
y = df["consumo_alto"]

X_treino, X_teste, y_treino, y_teste = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

preprocessamento = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), numericas),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categoricas)
])
modelo_consumo = Pipeline([
    ("preparacao", preprocessamento),
    ("arvore", DecisionTreeClassifier(max_depth=4, random_state=42))
])
modelo_consumo.fit(X_treino, y_treino)
previsoes = modelo_consumo.predict(X_teste)
print("Treino:", len(X_treino), "| Teste:", len(X_teste))
```

```
Treino: 4040 | Teste: 1010
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Avaliação do primeiro modelo Acurácia: proporção de previsões corretas. Precisão da classe alta: entre os turnos previstos como alto consumo, quantos de fato foram altos? Recall da classe alta: entre os turnos de alto consumo, quantos foram identificados? F1: equilíbrio entre precisão e recall. Uma matriz de confusão mostra acertos e erros. Em problemas reais, discuta qual tipo de erro é mais caro.

```python
def avaliar(nome, verdadeiro, previsto, rotulos=("Baixo", "Alto")):
    print(f"\n== {nome} ==")
    print(f"Acurácia: {accuracy_score(verdadeiro, previsto):.3f}")
    print(f"Precisão (classe 1): {precision_score(verdadeiro, previsto, zero_division=0):.3f}")
    print(f"Recall (classe 1): {recall_score(verdadeiro, previsto, zero_division=0):.3f}")
    print(f"F1 (classe 1): {f1_score(verdadeiro, previsto, zero_division=0):.3f}")
    print(classification_report(verdadeiro, previsto, target_names=rotulos, zero_division=0))
    ConfusionMatrixDisplay.from_predictions(verdadeiro, previsto, display_labels=rotulos, cmap="Blues")
    plt.title(nome); plt.show()

avaliar("Consumo ALTO/BAIXO", y_teste, previsoes)
```

```
== Consumo ALTO/BAIXO ==
Acurácia: 0.969
Precisão (classe 1): 0.974
Recall (classe 1): 0.960
F1 (classe 1): 0.967
              precision    recall  f1-score   support

       Baixo       0.96      0.98      0.97       533
        Alto       0.97      0.96      0.97       477

    accuracy                           0.97      1010
   macro avg       0.97      0.97      0.97      1010
```

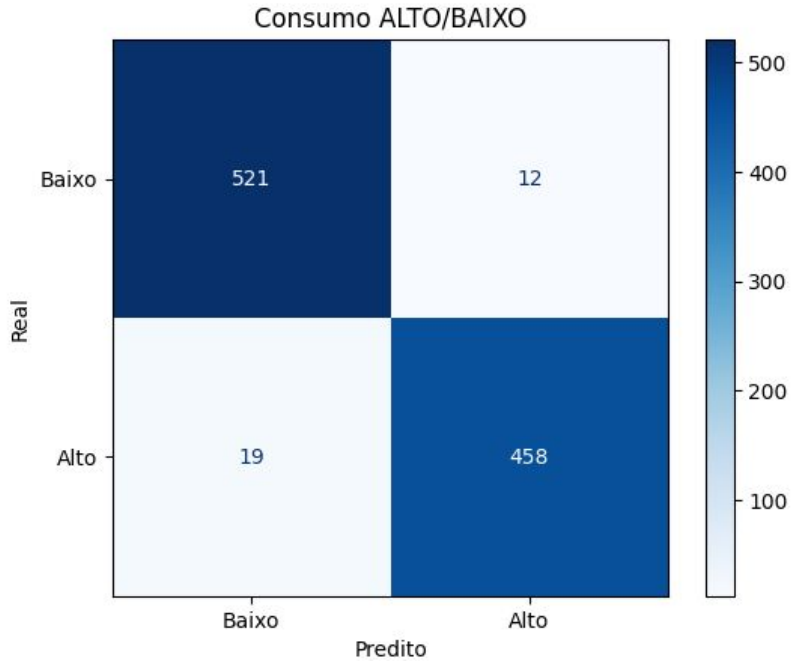
Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
== Consumo ALTO/BAIXO ==
Acurácia: 0.969
Precisão (classe 1): 0.974
Recall (classe 1): 0.960
F1 (classe 1): 0.967
              precision    recall  f1-score   support

       Baixo       0.96      0.98      0.97       533
        Alto       0.97      0.96      0.97       477

    accuracy                           0.97      1010
   macro avg       0.97      0.97      0.97      1010
weighted avg       0.97      0.97      0.97      1010
```



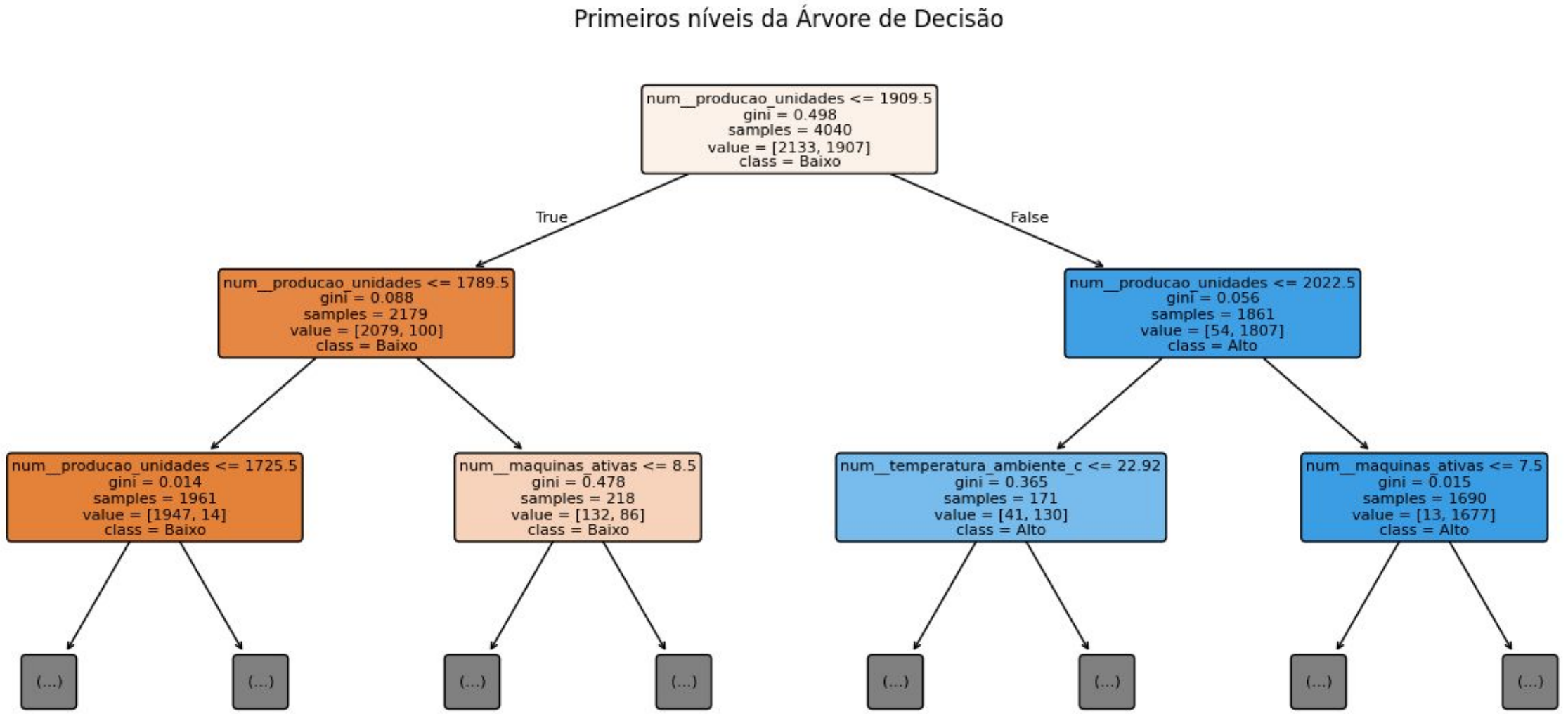
Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
# Visualização opcional: primeiros níveis da árvore
nomes = modelo_consumo.named_steps["preparacao"].get_feature_names_out()
plt.figure(figsize=(16,7))
plot_tree(modelo_consumo.named_steps["arvore"],
          feature_names=nomes, class_names=["Baixo", "Alto"],
          filled=True, rounded=True, fontsize=8, max_depth=2)
plt.title("Primeiros níveis da Árvore de Decisão"); plt.show()
```

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Segundo Classificador: Random Forest

Agora vamos treinar um modelo de **Random Forest** utilizando os mesmos dados de treino e a mesma estrutura de pré-processamento via `Pipeline`. Após o treinamento, utilizaremos a função `avaliar` para comparar as métricas com as da nossa primeira Árvore de Decisão.

```python
from sklearn.ensemble import RandomForestClassifier

# Criando o Pipeline com o preprocessamento existente e o classificador Random Forest
modelo_rf = Pipeline([
    ("preparacao", preprocessamento),
    ("floresta", RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42))
])

# Treinando o modelo
modelo_rf.fit(X_treino, y_treino)

# Fazendo as previsões no conjunto de teste
previsoes_rf = modelo_rf.predict(X_teste)

# Avaliando o desempenho do Random Forest
avaliar("Consumo ALTO/BAIXO (Random Forest)", y_teste, previsoes_rf)
```

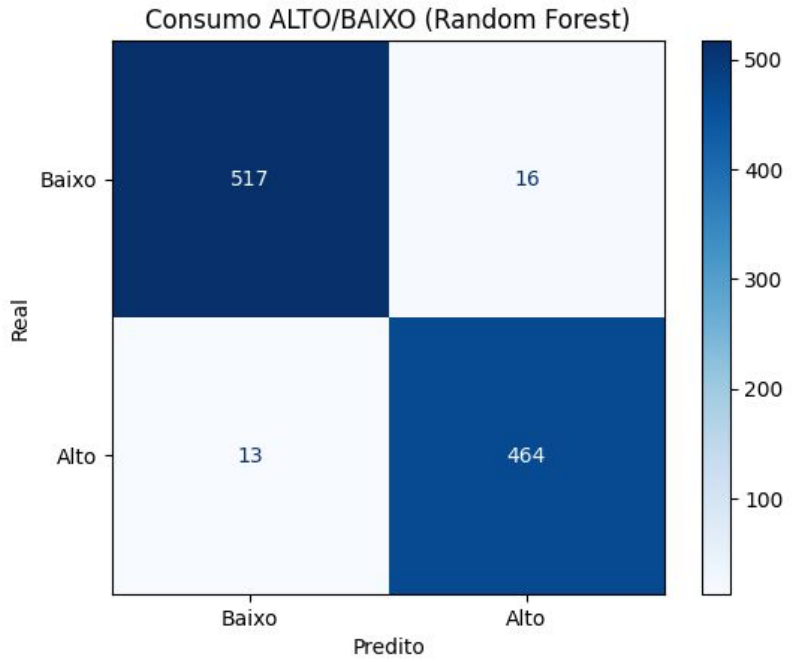
Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

```
== Consumo ALTO/BAIXO (Random Forest) ==
Acurácia: 0.971
Precisão (classe 1): 0.967
Recall (classe 1): 0.973
F1 (classe 1): 0.970
              precision    recall  f1-score   support

       Baixo       0.98      0.97      0.97       533
        Alto       0.97      0.97      0.97       477

    accuracy                           0.97      1010
   macro avg       0.97      0.97      0.97      1010
weighted avg       0.97      0.97      0.97      1010
```



Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

O modelo Random Forest foi avaliado utilizando **1.010 registros de teste**, classificados em consumo Alto ou Baixo.

- **Acurácia (97,1%):** aproximadamente 97% dos registros foram classificados corretamente.
- **Precisão (96,7%):** das previsões de consumo Alto, aproximadamente 97% estavam corretas.
- **Recall (97,3%):** o algoritmo identificou aproximadamente 97% dos registros que realmente apresentavam consumo Alto.
- **F1-score (97,0%):** indica equilíbrio entre precisão e recall.
- **Support:** 533 registros de consumo Baixo e 477 de consumo Alto.

Conclusão: o Random Forest apresentou elevado desempenho na classificação do consumo energético, com resultados semelhantes nas duas classes.

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Classificação do Consumo Energético — Random Forest

O algoritmo Random Forest foi utilizado para classificar o consumo energético industrial em duas categorias: **Alto (1) e Baixo (0)**.

Variáveis utilizadas: produção, temperatura ambiente, quantidade de máquinas ativas, horas de operação e turno.

Resultados obtidos:

- Acurácia: 97,1%.
- Precisão: 96,7%.
- Recall: 97,3%.
- F1-score: 97,0%.

Interpretação: o modelo apresentou elevado desempenho na classificação dos registros de teste, utilizando características operacionais da indústria para identificar padrões associados ao nível de consumo energético.

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

Comparação dos Modelos de Classificação

Os algoritmos Árvore de Decisão e Random Forest foram utilizados para classificar o consumo energético industrial em Alto e Baixo, a partir de variáveis operacionais.

Resultados:

- Árvore de Decisão: acurácia de 96,9%, com 31 classificações incorretas.
- Random Forest: acurácia de 97,1%, com 29 classificações incorretas.
- A Árvore de Decisão apresentou maior precisão na identificação de consumo Alto (97,4%).
- O Random Forest apresentou maior recall para consumo Alto (97,3%), identificando 464 dos 477 casos dessa categoria.

Conclusão: ambos os algoritmos apresentaram elevado desempenho no conjunto de teste. O Random Forest obteve uma pequena vantagem na acurácia e identificou mais registros de consumo Alto, enquanto a Árvore de Decisão produziu menos falsos alertas dessa categoria.

Exemplo 2 de Machine Learning e tratamento dos dados

Classificar as máquinas com base no consumo de energia

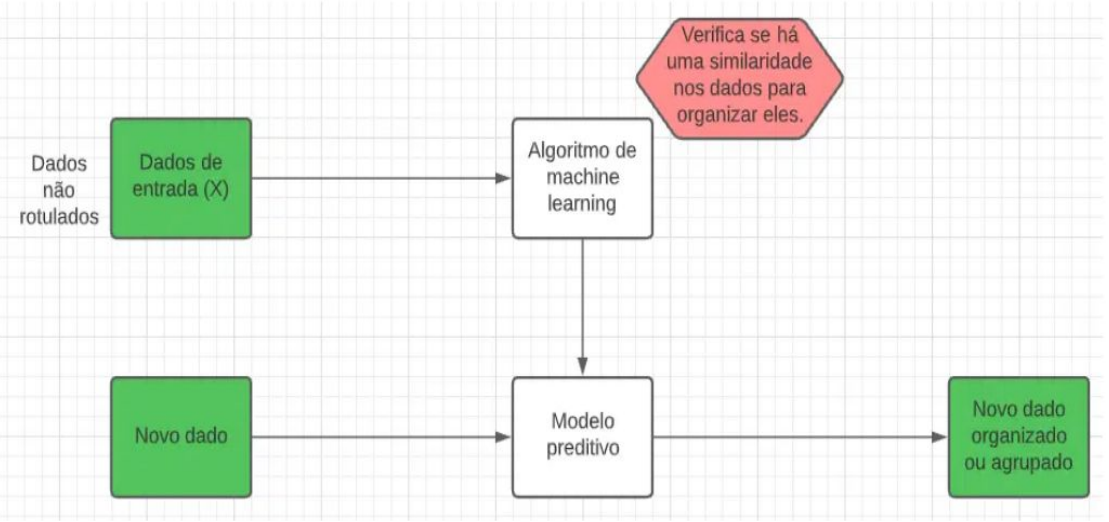
Em uma indústria, é mais prejudicial classificar um consumo baixo como alto ou deixar de identificar um consumo realmente alto?

Diferença entre programação tradicional e Machine Learning

- Programação tradicional:
- Dados + Regras programadas → Resultado
- Machine Learning:
- Dados + Resultados conhecidos → Treinamento → Modelo aprendido
- Depois:
- Novos dados + Modelo aprendido → Previsão

Aprendizado não supervisionado

É um algoritmo de aprendizado ocorre de forma independente,
ou seja, o algoritmo busca encontrar relações e semelhanças
entre os dados inseridos no algoritmo.



Aprendizado não supervisionado



Aprendizado não supervisionado

Exemplo 1: Uma indústria deseja analisar o desempenho de suas máquinas com base em dois indicadores: quantidade produzida e consumo de energia elétrica. A empresa coletou dados de nove máquinas, apresentados na tabela abaixo:

	maquina	producao	energia
0	M01	100	50
1	M02	120	55
2	M03	110	52
3	M04	500	200
4	M05	520	210
5	M06	490	195
6	M07	900	400
7	M08	920	410
8	M09	880	390

Aprendizado não supervisionado

Exemplo 1: Desenvolva um programa em Python utilizando as bibliotecas Pandas, Scikit-learn e Matplotlib para realizar as seguintes tarefas:

1. Crie um DataFrame com os dados de produção e consumo de energia das nove máquinas.
2. Utilize o `StandardScaler` para padronizar as variáveis antes de aplicar o algoritmo.
3. Implemente o algoritmo `KMeans`, considerando inicialmente **3 agrupamentos (K = 3)**.
4. Adicione ao DataFrame uma coluna chamada `grupo`, contendo o agrupamento atribuído a cada máquina.
5. Gere um gráfico de dispersão (`plt.scatter`) para visualizar os agrupamentos, utilizando produção no eixo X e consumo de energia no eixo Y.

Aprendizado não supervisionado

Exemplo 1: **Questões para análise**

a) Quantos grupos de máquinas foram identificados pelo algoritmo?

b) Quais máquinas apresentam características semelhantes de produção e consumo energético?

c) Qual grupo apresenta maior produção e maior consumo de energia?

d) Uma máquina que apresenta maior produção necessariamente possui melhor eficiência energética? Justifique.

e) Por que é importante padronizar os dados antes de aplicar o K-Means?

Desafio adicional: Utilize o método do cotovelo (*Elbow Method*) para avaliar se K = 3 é uma escolha adequada para esse conjunto de dados. Considere os valores de K entre 1 e 8.

Aprendizado não supervisionado - Código

```
1
2 # Importa a biblioteca Pandas para criar e manipular tabelas
3 import pandas as pd
4
5 # Importa o Matplotlib para criar gráficos
6 import matplotlib.pyplot as plt
7
8 # Importa o algoritmo K-Means para realizar agrupamentos
9 from sklearn.cluster import KMeans
10
11 # Importa o StandardScaler para padronizar os dados
12 from sklearn.preprocessing import StandardScaler
```

Aprendizado não supervisionado - Código

```
# ========================================
# 1. CRIAÇÃO DO CONJUNTO DE DADOS
# ========================================

# Dados coletados de nove máquinas industriais
# Cada máquina possui informações de produção e
# consumo de energia elétrica

dados = {
    "maquina": ["M01", "M02", "M03",
                "M04", "M05", "M06",
                "M07", "M08", "M09"],

    "producao": [100, 120, 110, 500, 520, 490,
                 900, 920, 880],

    "energia": [50, 55, 52, 200, 210, 195,
                400, 410, 390]
}

# Converte o dicionário em um DataFrame
df = pd.DataFrame(dados)

# Exibe os dados originais
print("Dados das máquinas:")
print(df)
```

Aprendizado não supervisionado - Código

```
# ========================================
# 2. PADRONIZAÇÃO DOS DADOS
# ========================================

# O K-Means utiliza distâncias para agrupar os dados
# Por isso, variáveis com escalas diferentes podem
# influenciar de maneira desigual os agrupamentos

# Cria o objeto responsável pela padronização
scaler = StandardScaler()

# Seleciona as variáveis utilizadas pelo K-Means
# fit_transform():
# fit: calcula a média e o desvio-padrão
# transform: padroniza os valores
#
# Após a padronização:
# Média aproximadamente igual a 0
# Desvio-padrão igual a 1

X = scaler.fit_transform(df[["producao", "energia"]])

# Exibe os dados padronizados
print("\nDados padronizados:")
print(X)

```

Aprendizado não supervisionado - Código

```
28 # ==========================================
29 # 3. CRIAÇÃO DO MODELO K-MEANS
30 # ==========================================
31
32 # Cria o algoritmo de agrupamento K-Means
33 modelo = KMeans(
34     n_clusters=3,    # Quantidade de grupos desejados (K=3)
35     random_state=42, # Permite reproduzir os resultados
36     n_init=10         # Executa o K-Means 10 vezes com
37                       # diferentes inicializações e
38                       # seleciona o resultado de menor inércia
39 )
40
41
42 # ==========================================
43 # 4. TREINAMENTO E IDENTIFICAÇÃO DOS GRUPOS
44 # ==========================================
45
46 # fit_predict(X):
47 # fit: treina o algoritmo utilizando os dados padronizados
48 # predict: identifica a qual grupo cada máquina pertence
49 #
50 # Os grupos são identificados por números: 0, 1 e 2
51 # Esses números não representam uma ordem de desempenho
```

Aprendizado não supervisionado - Código

```
42 # ========================================
43 # 4. TREINAMENTO E IDENTIFICAÇÃO DOS GRUPOS
44 # ========================================
45
46 # fit_predict(X):
47 # fit: treina o algoritmo utilizando os dados padronizados
48 # predict: identifica a qual grupo cada máquina pertence
49 #
50 # Os grupos são identificados por números: 0, 1 e 2
51 # Esses números não representam uma ordem de desempenho
52
53 df["grupo"] = modelo.fit_predict(X)
54
```

Aprendizado não supervisionado - Código

```
# ========================================
# 5. CÁLCULO DA EFICIÊNCIA ENERGÉTICA
# ========================================

# A eficiência energética indica quantas unidades
# são produzidas para cada kWh consumido
#
# Fórmula:
# Eficiência = Produção / Energia
#
# Quanto maior o resultado, maior a quantidade
# produzida por unidade de energia consumida

df["eficiencia"] = df["producao"] / df["energia"]

# Exibe as máquinas organizadas por grupo
print("\nResultado dos agrupamentos:")
print(df.sort_values("grupo"))


```

Aprendizado não supervisionado - Código

```
76 # ========================================
77 # 6. VISUALIZAÇÃO DOS AGRUPAMENTOS
78 # ========================================
79
80 plt.figure(figsize=(10, 8))
81
82 plt.scatter(
83     df["producao"],
84     df["energia"],
85     c=df["grupo"],
86     cmap="viridis",
87     s=120
88 )
89
```

Aprendizado não supervisionado - Código

```
90 # Deslocamentos manuais para evitar sobreposição dos textos
91 deslocamentos = {
92     "M01": (8, 5),
93     "M02": (8, 10),
94     "M03": (8, -10),
95     "M04": (8, 5),
96     "M05": (8, 10),
97     "M06": (8, -10),
98     "M07": (8, 5),
99     "M08": (8, 10),
100    "M09": (8, -10)
101 }
102
```

Aprendizado não supervisionado - Código

```
103 # Adiciona o nome de cada máquina com deslocamento
104 for i in range(len(df)):
105     maquina = df.loc[i, "maquina"]
106     x = df.loc[i, "producao"]
107     y = df.loc[i, "energia"]
108 
109     dx, dy = deslocamentos[maquina]
110 
111     plt.annotate(
112         maquina,
113         (x, y),
114         xytext=(dx, dy),
115         textcoords="offset points",
116         fontsize=10,
117         bbox=dict(boxstyle="round,pad=0.2", fc="white", alpha=0.7)
118     )
119 
120 plt.xlabel("Produção (unidades)")
121 plt.ylabel("Energia (kWh)")
122 plt.title("Agrupamento de Máquinas - K-Means")
123 plt.grid(alpha=0.3)
124 plt.show()
```

Aprendizado não supervisionado - Código

```
125 # ========================================
126 # 7. MÉTODO DO COTOVELO (ELBOW METHOD)
127 # ========================================
128
129 # O método do cotovelo ajuda a avaliar
130 # a quantidade adequada de grupos (K)
131 #
132 # Ele utiliza a inércia (WCSS), que representa
133 # a soma das distâncias quadráticas dos pontos
134 # aos centroides de seus respectivos grupos
135 #
136 # Quanto menor a inércia, mais próximos os pontos
137 # estão dos centroides de seus grupos
138
139 # Cria uma lista vazia para armazenar as inércias
140 inercias = []
141
142 # Testa diferentes quantidades de grupos
143 # range(1, 9) gera valores de K entre 1 e 8
144 for k in range(1, 9):
```

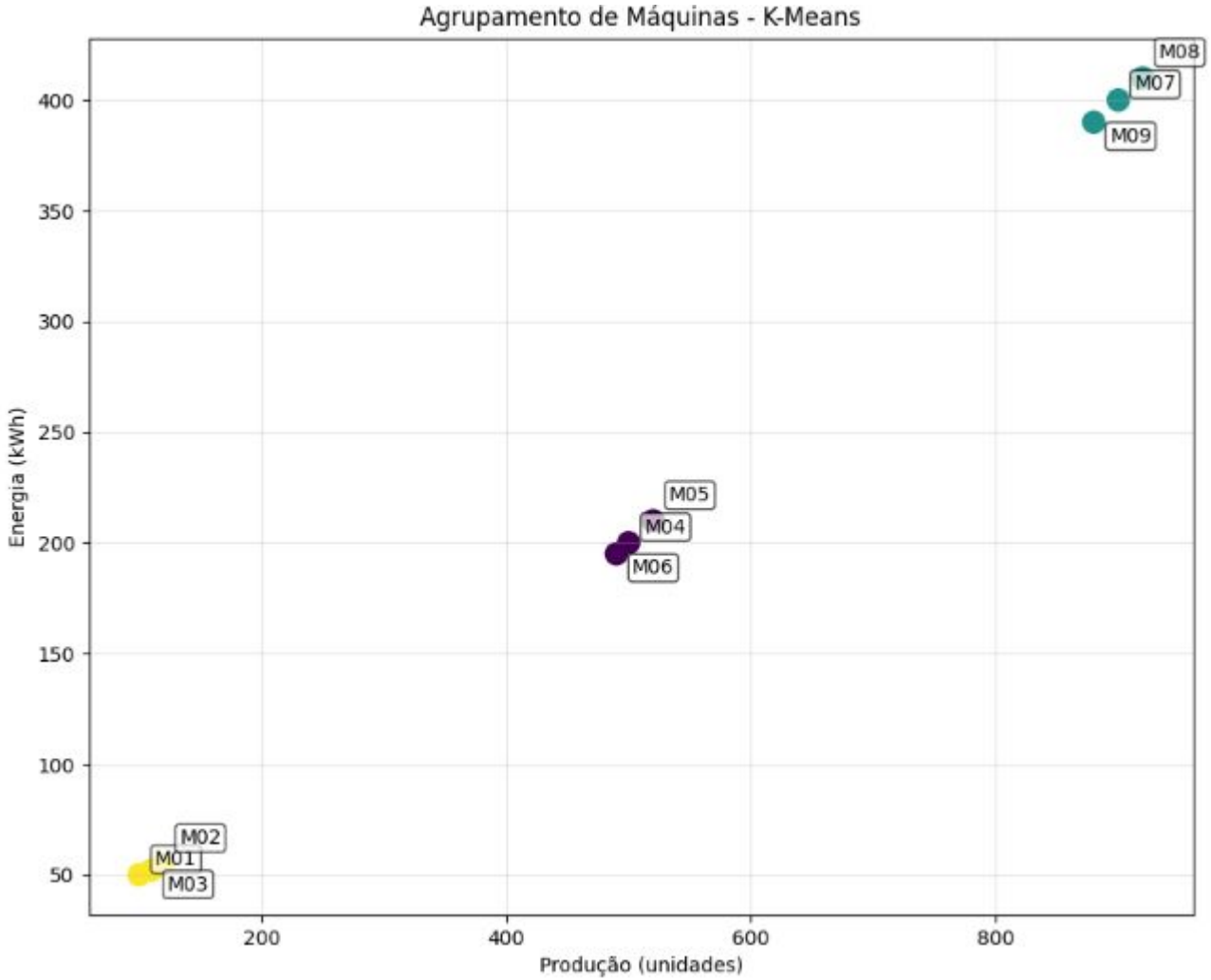
Aprendizado não supervisionado - Código

```
142 # Testa diferentes quantidades de grupos
143 # range(1, 9) gera valores de K entre 1 e 8
144 for k in range(1, 9):
145 
146     # Cria um novo modelo K-Means para cada valor de K
147     modelo_k = KMeans(
148         n_clusters=k,      # Quantidade de grupos testada
149         random_state=42,   # Mantém a reprodutibilidade
150         n_init=10          # Testa 10 inicializações
151     )
152 
153     # Treina o modelo com os dados padronizados
154     modelo_k.fit(X)
155 
156     # inertia_ retorna a inércia do modelo treinado
157     # append() adiciona esse valor à lista
158     inercias.append(modelo_k.inertia_)
159 
160 
161 
162 
```

Aprendizado não supervisionado - Código

```
Resultado dos agrupamentos:
  maquina  producao  energia  grupo  eficiencia
3     M04       500      200      0    2.500000
4     M05       520      210      0    2.476190
5     M06       490      195      0    2.512821
6     M07       900      400      1    2.250000
7     M08       920      410      1    2.243902
8     M09       880      390      1    2.256410
0     M01       100       50      2    2.000000
2     M03       110       52      2    2.115385
1     M02       120       55      2    2.181818
```

Aprendizado não supervisionado - Código



Aprendizado não supervisionado - Código

```python
# ========================================
# 8. GRÁFICO DO MÉTODO DO COTOVELO
# ========================================

# Define o tamanho do gráfico
plt.figure(figsize=(8, 5))

# Cria um gráfico de linha
# Eixo X: quantidade de grupos (K)
# Eixo Y: valor da inércia
plt.plot(
    range(1, 9),
    inercias,
    marker="o"      # Exibe um marcador em cada valor de K
)

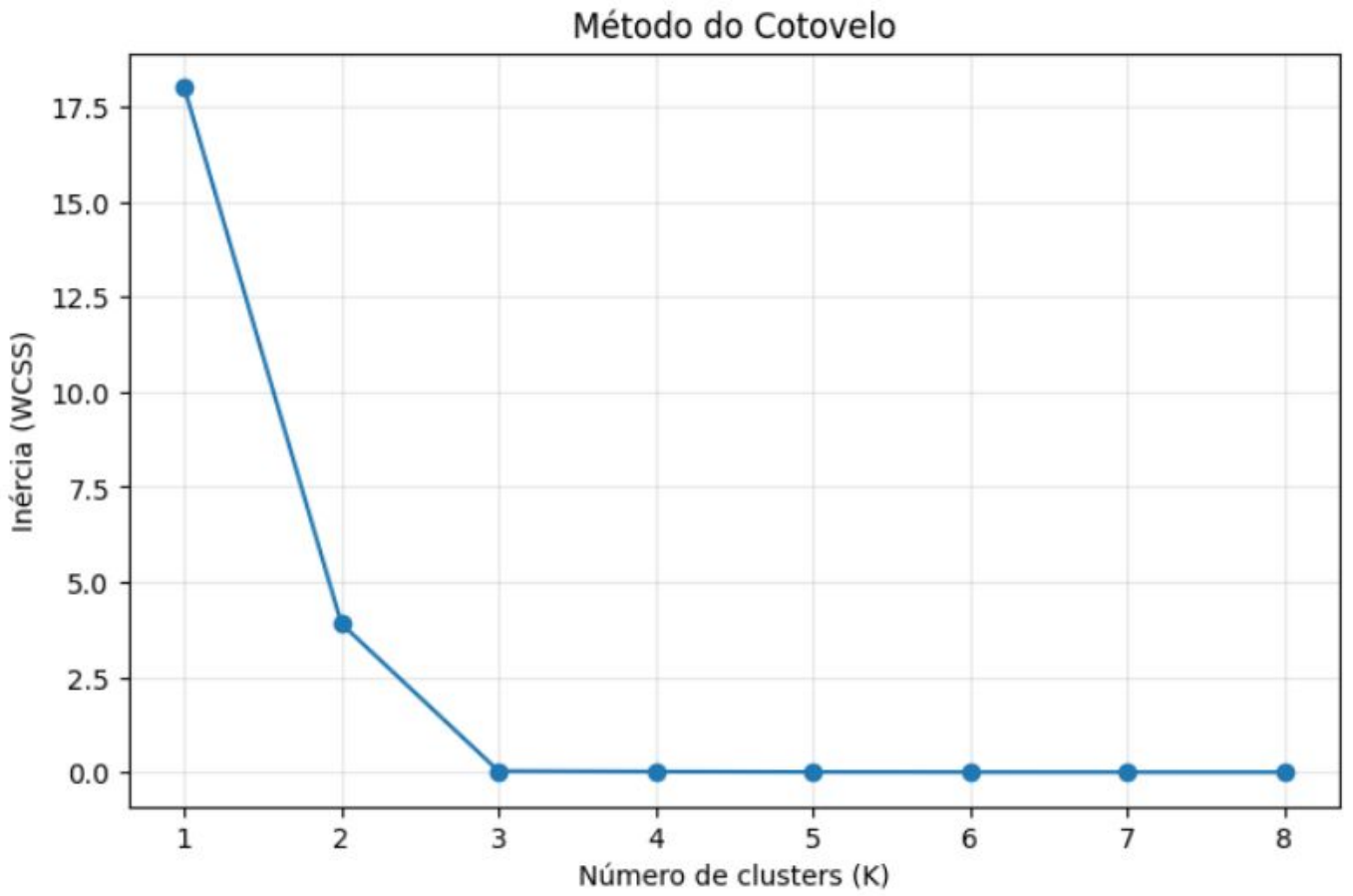
# Define os nomes dos eixos
plt.xlabel("Número de clusters (K)")
plt.ylabel("Inércia (WCSS)")

# Define o título do gráfico
plt.title("Método do Cotovelo")

# Exibe todos os valores de K no eixo X
plt.xticks(range(1, 9))

# Adiciona uma grade
plt.grid(alpha=0.3)

# Exibe o gráfico
plt.show()
```



Aprendizado não supervisionado - Inertia

WCSS – Within-Cluster Sum of Squares

O WCSS (Soma dos Quadrados Dentro dos Clusters) é uma métrica utilizada no algoritmo K-Means para medir o grau de proximidade entre os pontos de cada grupo e seu respectivo centroide.

O WCSS calcula a soma dos quadrados das distâncias entre cada ponto e o centroide do cluster ao qual pertence.

$$WCSS = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$$

Em que:

- K: número de clusters.
- C_i: conjunto de pontos do cluster i.
- x: ponto pertencente ao cluster.
- μ_i: centroide do cluster.

Aprendizado não supervisionado - Inertia

Em que:

- K: número de clusters.
- C_i: conjunto de pontos pertencentes ao cluster i.
- x: ponto de dados.
- μ_i: centroide do cluster i.
- $\|x - \mu_i\|^2$: distância euclidiana ao quadrado entre o ponto e o centroide.

Relação com o método do cotovelo

O método do cotovelo utiliza o WCSS para determinar uma quantidade adequada de clusters.

- WCSS alto: os pontos estão mais distantes dos centroides, indicando maior dispersão dentro dos grupos.
- WCSS baixo: os pontos estão mais próximos dos centroides, indicando grupos mais compactos.
- Ponto do cotovelo: região do gráfico em que aumentar o número de clusters deixa de produzir reduções expressivas no WCSS.

Aprendizado não supervisionado - Inertia

Quanto menor a inércia, mais próximos os pontos estão de seus centroides. Entretanto, aumentar K sempre reduz ou mantém a inércia, por isso buscamos o ponto em que a redução começa a perder

Aprendizado não supervisionado Exercícios

Exercício 1 – Agrupamento de Viagens utilizando K-Means

Uma empresa de transportes deseja analisar o perfil de suas viagens para identificar padrões de deslocamento e auxiliar no planejamento logístico.

Para isso, foram coletados dados referentes à distância percorrida (km) e ao tempo de viagem (horas) de 12 viagens realizadas.

A empresa deseja utilizar o algoritmo K-Means para identificar grupos de viagens com características semelhantes, sem definir previamente a categoria de cada viagem.

Aprendizado não supervisionado Exercício 1

Viagem	Distância (km)	Tempo (horas)
V01	15	0,5
V02	20	0,7
V03	18	0,6
V04	25	0,8
V05	150	2,5
V06	180	3,0
V07	160	2,8
V08	200	3,5
V09	650	9,0
V10	700	10,0
V11	680	9,5
V12	750	11,0

Aprendizado não supervisionado - Exercício 1

Desenvolva um programa em Python utilizando Pandas, Scikit-learn e Matplotlib para:

1. Criar um DataFrame contendo os dados das viagens.
2. Selecionar as variáveis `distancia` e `tempo`.
3. Padronizar os dados utilizando `StandardScaler`.
4. Aplicar o algoritmo `KMeans` considerando **K = 3**.
5. Adicionar ao DataFrame uma coluna chamada `grupo`, identificando o agrupamento de cada viagem.
6. Criar um gráfico de dispersão (`plt.scatter`) com distância no eixo X e tempo no eixo Y, diferenciando os grupos por cores.
7. Utilizar o método do cotovelo para analisar a escolha do número de agrupamentos.

Aprendizado não supervisionado - Exercício 1

Questões para análise

a) Quantos grupos de viagens foram identificados?

b) Quais viagens apresentam características semelhantes?

c) Qual grupo apresenta as maiores distâncias e os maiores tempos de deslocamento?

d) Como esses agrupamentos podem auxiliar no planejamento logístico da empresa?

e) O método do cotovelo indica que K = 3 é uma escolha adequada? Justifique.

Aprendizado não supervisionado - Exercício 1

Entregar através do Link:

Exercício 1

Aprendizado não supervisionado - Exercício 2

Exercício 2 – Agrupamento de Máquinas Industriais com Vibração e Corrente Elétrica

Uma indústria utiliza sensores para monitorar as condições operacionais de suas máquinas. Entre os parâmetros monitorados estão a vibração mecânica (mm/s) e a corrente elétrica consumida (A).

A equipe de manutenção deseja identificar máquinas com comportamentos semelhantes e verificar a existência de grupos que apresentem valores elevados dessas variáveis.

Como não existem classificações previamente definidas, será utilizado o algoritmo K-Means para identificar diferentes perfis operacionais.

Aprendizado não supervisionado - Exercício 2

Base de dados

Máquina	Vibração (mm/s)	Corrente (A)
M01	1,2	5,0
M02	1,5	5,5
M03	1,3	5,2
M04	1,7	6,0
M05	3,5	10,0
M06	3,8	11,0
M07	4,0	10,5
M08	4,2	12,0
M09	7,5	18,0
M10	8,0	19,0
M11	8,5	20,0
M12	9,0	21,0

Aprendizado não supervisionado - Exercício 2

Desenvolva um programa em Python utilizando Pandas, Scikit-learn e Matplotlib para:

1. Criar um DataFrame contendo os dados das máquinas.
2. Selecionar as variáveis `vibracao` e `corrente`.
3. Padronizar os dados utilizando `StandardScaler`.
4. Implementar o algoritmo `KMeans` considerando **K = 3**.
5. Adicionar ao DataFrame uma coluna chamada `grupo`.
6. Criar um gráfico de dispersão (`plt.scatter`) com vibração no eixo X e corrente elétrica no eixo Y, utilizando cores diferentes para representar os grupos.
7. Aplicar o método do cotovelo para avaliar a escolha do número de agrupamentos.

Aprendizado não supervisionado - Exercício 2

Questões para análise

a) Quantos grupos de máquinas foram identificados?

b) Quais máquinas apresentam os menores valores de vibração e corrente elétrica?

c) Qual grupo apresenta os maiores valores dessas variáveis?

d) Os agrupamentos podem ajudar a equipe de manutenção a identificar máquinas que precisam de inspeção? Explique.

e) Uma máquina com vibração e corrente elevadas necessariamente apresenta uma falha? Justifique.

f) O método do cotovelo confirma K = 3 como uma escolha adequada?

Aprendizado não supervisionado - Exercício 2

Exercício 2

Link material

Material disciplina

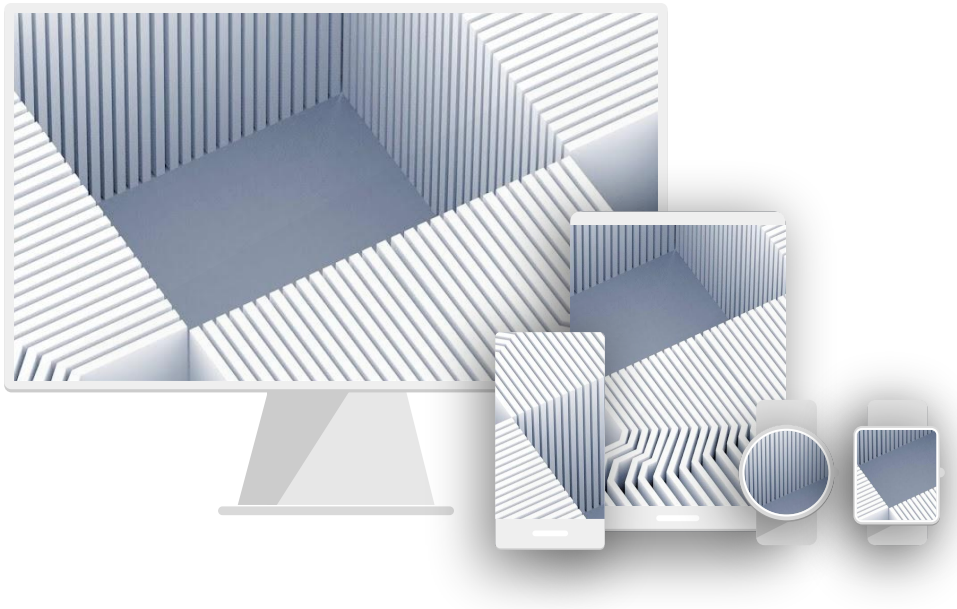
Obrigado !

Prof. Me Daniel Vieira

Email: danielvieira2006@gmail.com

Linkedin: Daniel Vieira

Instagram: Prof daniel.vieira95



UNISENAI
Inovação no
Ensino Superior
para a Indústria